Student Exchange | Progress Review

A working account/admin foundation, a polished preview, and a clear integration backlog

BUY. SELL. CONNECT.

From a backend repository to a visible student marketplace experience.

Students reuse affordable campus essentials. Administrators maintain accounts, schools, access and abuse controls.

24 OCT

2026

Technical checkpoint: implementation + evidence + next gates

Tristan Roa • Robert (Andrew) Quaife • Alex Pyankov

Robert Richardson • Thomas Phommarath

Design understanding: goals → requests → identity → persistence → evidence.

Current status: the foundation works; the market is still a preview

Separate implementation, fixture verification, browser behavior and live deployment claims

2 / 2

Application builds pass

Frontend typecheck + Vite build; .NET 9 build: zero warnings/errors.

116

Tests passed

One failing accessibility expectation; one live test intentionally skipped.

PREVIEW

Marketplace boundary

Search, detail, messages and offers use sample/local browser state.

Integration implemented

Registration, login, profile, logout and administration use Node → ASP.NET contracts; supplied screenshots show deployed admin views.

Final integration gate

Connect authenticated, persistent listings; verify the service-token contract on the deployed stack.

Implemented and tested against fixtures does not establish current production deployment readiness.

Fresh build/test logs; browser-checks.json; Website/docs/live-validation.md

Problem, users and project goals

A campus exchange workflow supported by account identity and administrative control

Students

Discover items by category or query; inspect details; manage a school-linked account; eventually connect with a real seller.

Administrators

Maintain users, schools and admin membership; inspect/clear attempt records; request target session sign-out.

Delivery stakeholders

A consistent implementation, explainable design, traceable contributions, test evidence and a credible final demonstration.

Current boundary: web UI + Node adapter + ASP.NET API + SQLite. Mobile web is ongoing. Payment processing and live seller delivery are outside the implemented flow.

The product goal is a usable student exchange; the demonstrated catalog currently contains six illustrative records.

Website/src/App.tsx; data/listings.ts; admin.mjs; team role input

What changed: a verified repository comparison

Proposal comparison is limited because the original proposal artifact and date were not supplied

11 Oct · repository baseline

Backend and API documentation present. This is the available comparison point, not a verified proposal baseline.

24 Oct · source update

Website + Node adapter + admin console + tests added; listing endpoint and schema/config refinements added.

24 Oct · design artifacts

25 UML diagrams and 18 DFDs now document current data, interaction and deployment boundaries.

Plan implication	Observed adjustment / constraint
Browser/backend contract	Node adapter handles backend GET bodies and protects credentials.
Marketplace breadth	Visible messaging/offer UI exists before live listing persistence.
Role boundaries	Andrew reports website work in addition to his original backend role.

Proposal-date scope comparison remains pending; repository changes and team-delivered artifacts are documented.

Functional requirements: concrete acceptance and status

Status describes the current source and evidence, rather than an estimated completion percentage

ID / capability	Acceptance condition	Current status
FR1 · discovery	Filter by query/category and open details.	Local preview verified: 6 cards → 3 textbooks.
FR2 · accounts	Register; login; restore/edit profile; logout.	Implemented; local HTTP/UI fixtures pass.
FR3 · administration	Enforce levels and valid target scope.	Implemented; admin UI + wire fixtures pass.
FR4 · listings	Create/read/delete persisted, owned listings.	Direct API exists; website + schema integration missing.
FR5 · communication	Send durable messages/offers to a real seller.	Messaging/offer UI only; no delivery or storage.
FR6 · mobile web	Usable account/marketplace experience on phones.	Work ongoing; 390 px home-page check passes.

A visible control is not a completed end-to-end feature unless its server and retained-data path exist.

Source routes; test-summary.json; browser-checks.json; PROCESS-SPECS.md

Non-functional requirements: evidence and remaining limits

Observed controls are separated from acceptance work still needed

Requirement	Implemented / observed	Remaining acceptance
Credential isolation	Node-only tokens; HttpOnly cookie; field projection.	Replace reversible password storage; verify deployed behavior.
Data integrity	Unique account email/token; declared relationships.	Versioned Listings schema; explicit FK enforcement check.
Reliability	Safe upstream errors; saved-write/readback handling.	Deployed migration smoke test; restart/session expectations.
Usability / accessibility	Keyboard/card behavior; responsive home at 390 px.	Repair accessible-list label; audit account/admin/mobile views.
Performance / operations	Bounded upstream deadlines; /healthz process check.	Measure latency/load; verify service auth and DB readiness.

Security, accessibility, integrity and deployment readiness still need explicit final acceptance checks.

Completed implementation: what is actually present

Feature delivery, integration evidence and remaining edge are explicit

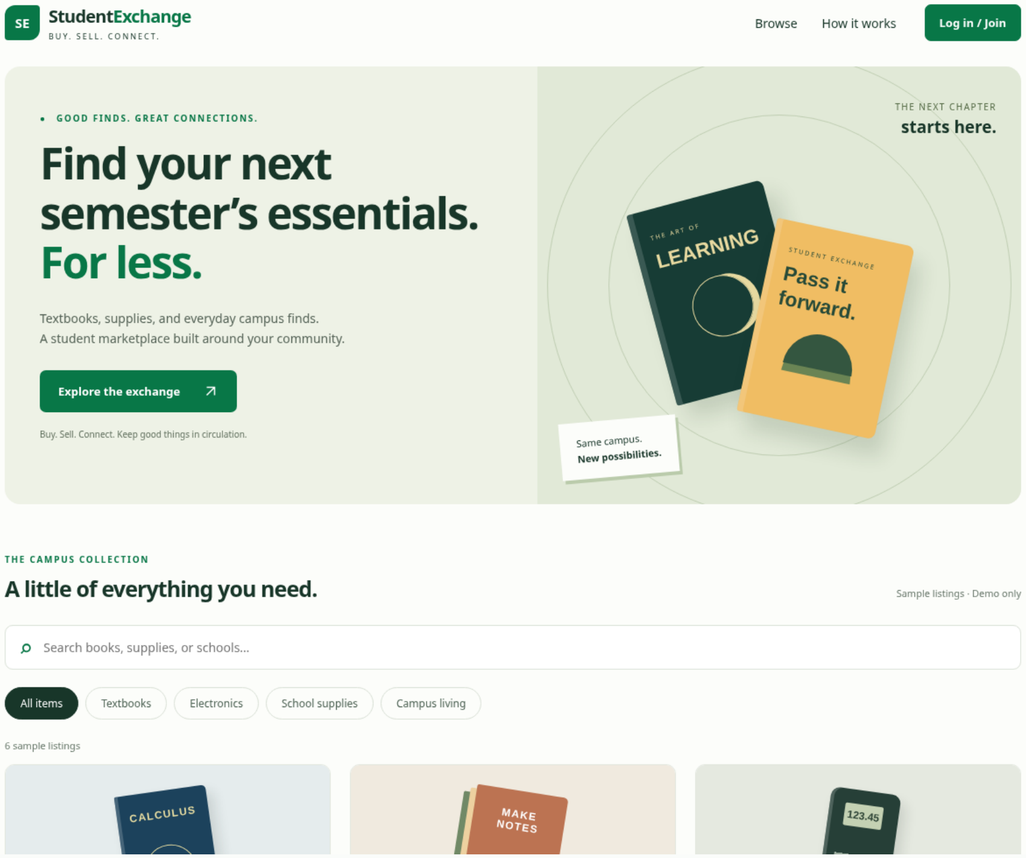
Workstream	Completed artifact / behavior	What remains
Frontend	Browse/detail UI; account forms; admin console; local messaging.	Live marketplace and full mobile/accessibility acceptance.
Backend / API	Account/profile/token, school, role and block-management operations.	Listing auth/ownership and deployment acceptance.
Adapter	Same-origin routes; session Map; service-token forwarding.	Current Azure/Render smoke test and production readiness.
Data / infrastructure	Eight-table DDL; configurable account DB; Render manifest.	Listings migration, path alignment, FK/runtime checks.
Design / verification	UML + DFD atlases; fresh builds and 116 passing cases.	One failed assertion, offer input issue, final acceptance run.

The strongest completed vertical slice is account and administrator management; persistent marketplace activity is the main remaining slice.

Website/src and server; Backend/Controllers and Components; user contribution report

Website evidence: the current frontend running locally

Captured from the current production bundle served by the Node website server



Observed in Chromium

6 sample listings render

3 remain after Textbooks

No browser script errors

390 px home: no overflow

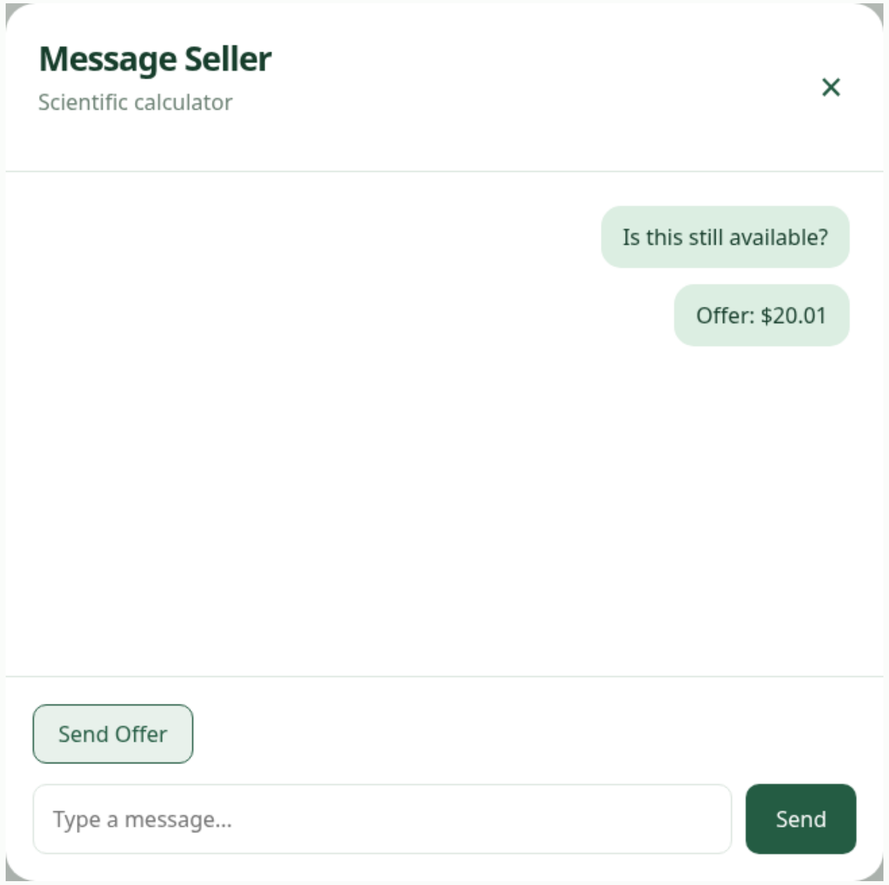
Accounts/admin use fixture evidence here; no new live backend run is claimed.

The UI is demonstrable today; screenshot evidence is scoped to local preview behavior.

Local Chromium capture; npm run build; assets/website-home.png

Messaging UI: local interaction and the remaining delivery gap

Robert Richardson’s messaging UI changes local App state



What works

Open message dialog; append a local message; preview a positive offer.

What is missing

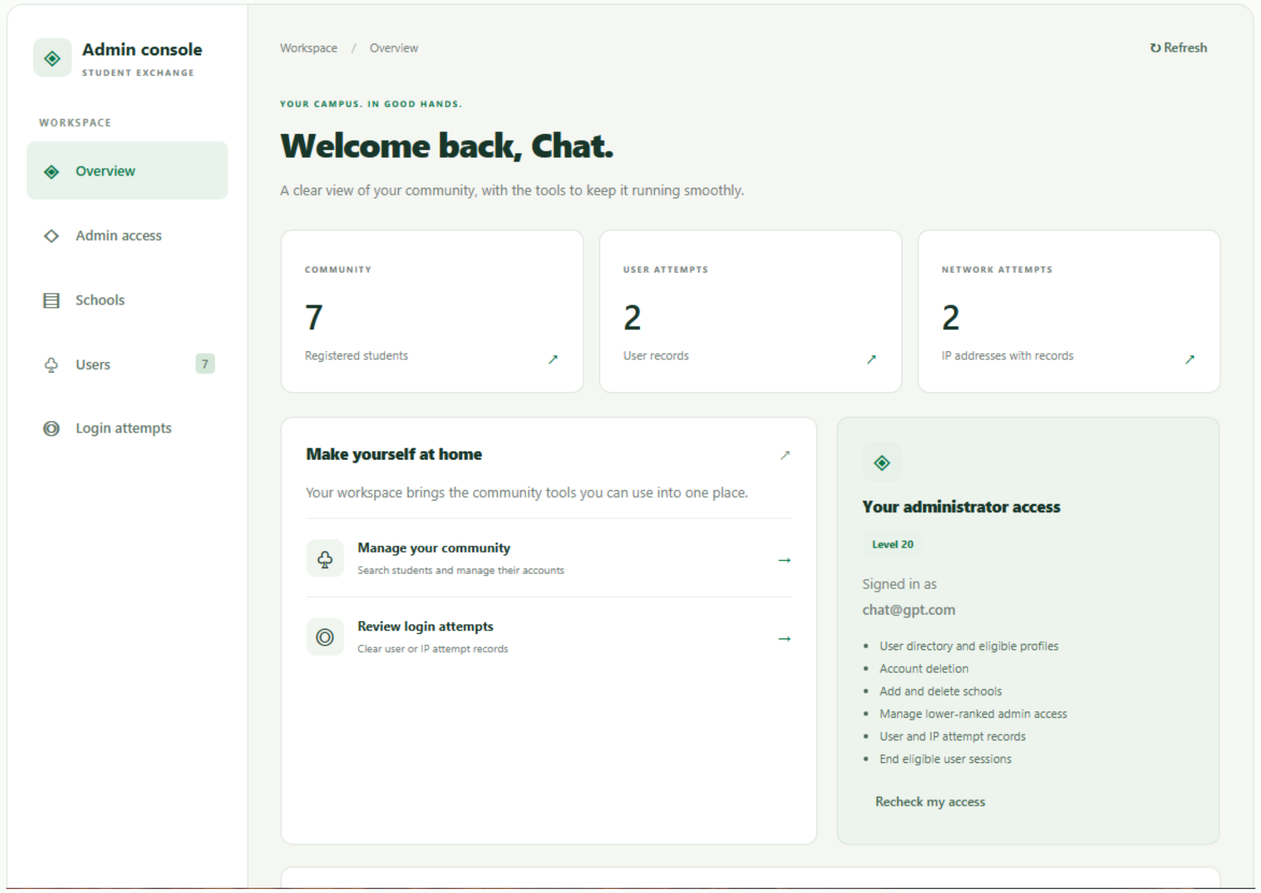
Seller delivery and persistence. Fix whole-dollar step validation and isolate conversation state by listing.

Next integration contract: authenticated sender + seller/listing identity + durable message/offer records.

App.tsx form handlers; local browser capture; team-reported attribution

Deployed admin console: overview and access context

Team-supplied capture from studentexchange.onrender.com/admin



7 students displayed

Snapshot count; not a permanent project metric.

2 user + 2 IP records

Attempt observations; not necessarily active blocks.

Level 20 shown

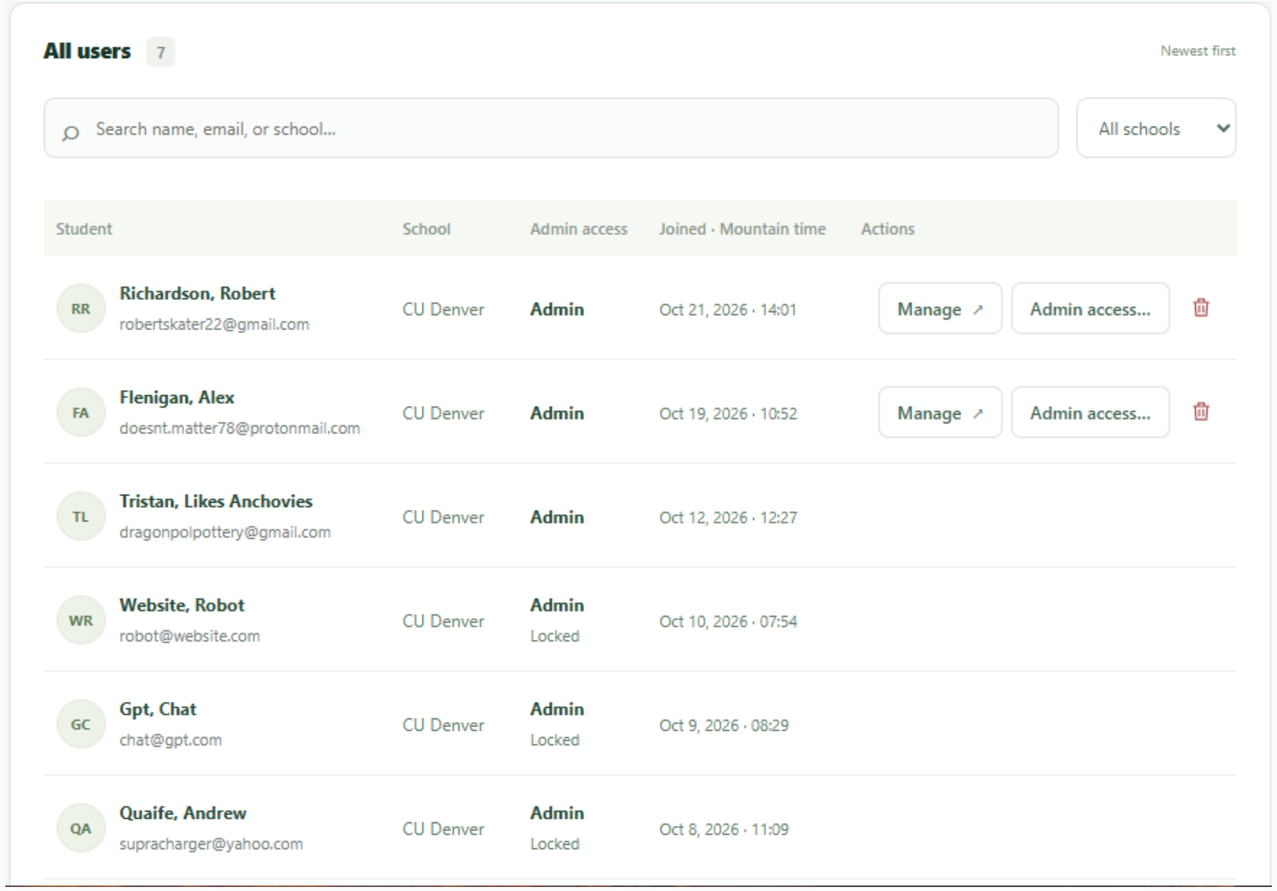
Permission context is visible in the supplied capture.

The deployed console exposes community and attempt information under a visible administrator context.

User-provided admin overview screenshot; team-confirmed Render deployment

Deployed user directory: visible management capabilities

Team-supplied Users view with search, school filter, role/lock indicators and actions



Find and filter

Search by name/email/school; school dropdown.

Review access context

Admin/locked indicators and joined timestamps.

Manage eligible targets

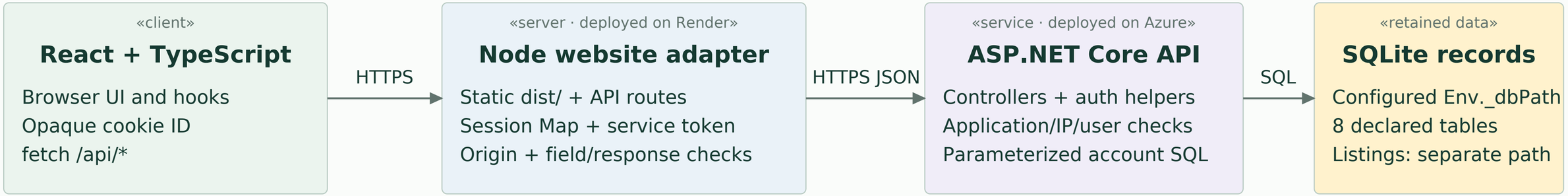
Manage, admin-access and delete controls are visible.

The directory screenshot, source contracts and passing admin fixtures provide complementary evidence.

User-provided admin Users screenshot; server/admin.mjs and api.mjs; backend Users controller

Current architecture: browser → adapter → API → data

A same-origin website adapter bridges a body-authenticated backend API



Identity boundary

Browser: cookie ID. Node: visitor access_token + service admin_token. Backend: caller and selected target remain distinct.

Independent listing path

Direct /api/Listing CRUD is present, but it is not invoked by the website adapter and lacks a supplied table migration.

The adapter is necessary for both protocol compatibility and keeping shared/user credentials out of the browser.

Website/server/app.mjs, api.mjs, render.yaml; Backend/Program.cs, Sql.cs, Env.cs, Listing.cs

Walk one request: reading a signed-in profile

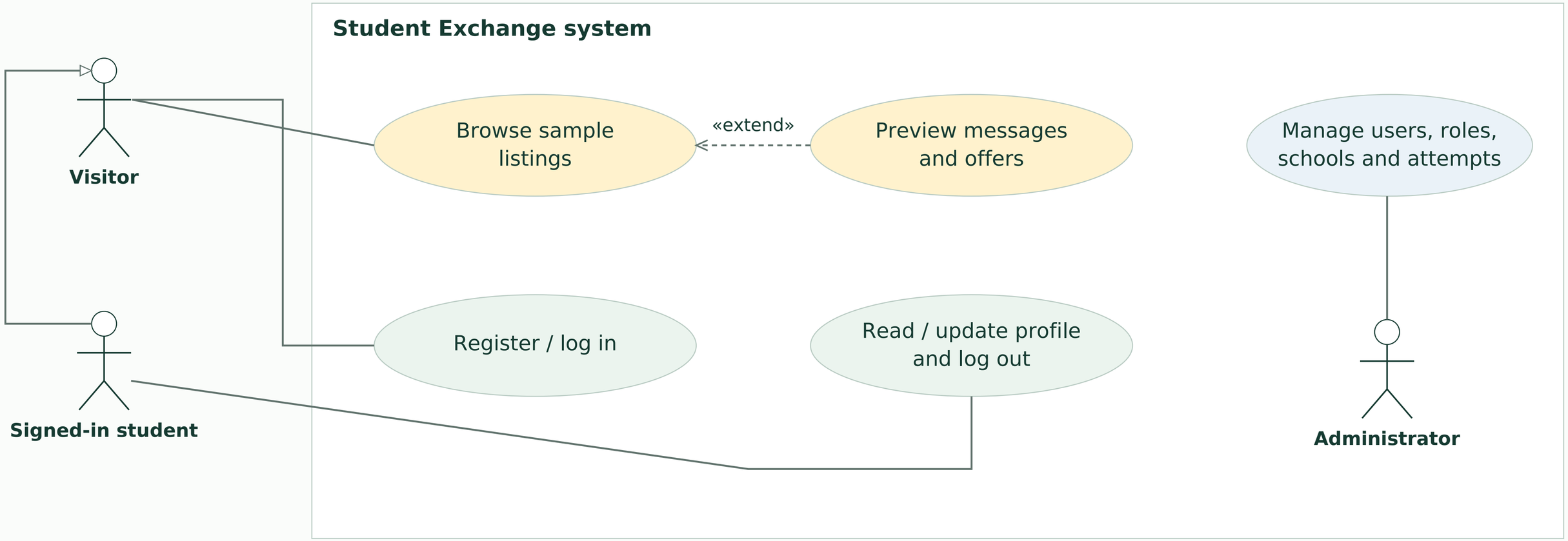
How the request, identity and data move across our layers

1 · Browser	GET /api/profile + opaque cookie; no credentials in localStorage.
2 · Node adapter	Cookie ID → session Map → visitor token; add server-owned envelope.
3 · ASP.NET + SQLite	Validate app/IP/token; select user and school; produce profile data.
4 · Browser-safe result	Return email, first_name, last_name, school; expire true invalid sessions.

GET bodies are an upstream contract constraint; the browser-side contract remains a normal same-origin request.

UML use cases: goals and implemented boundaries

The important distinction is preview interaction versus account/admin integration

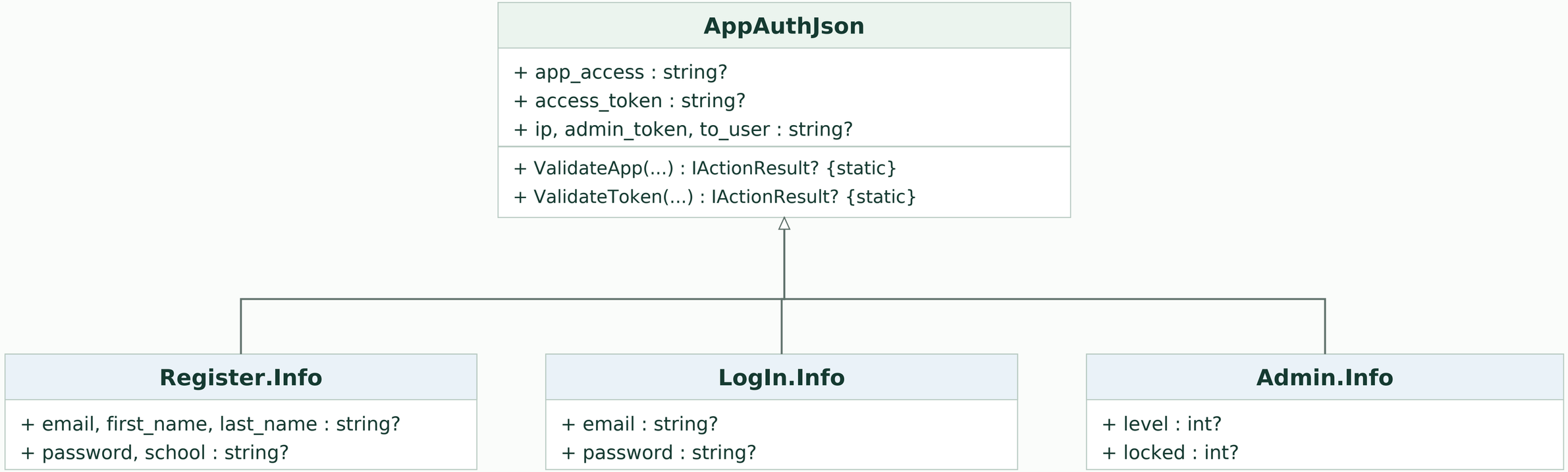


Use cases describe user goals; backend service identity is infrastructure rather than a human admin actor.

Adapted from docs/uml D02/D03; source App.tsx and server/api.mjs

UML class view: shared credentials and request inheritance

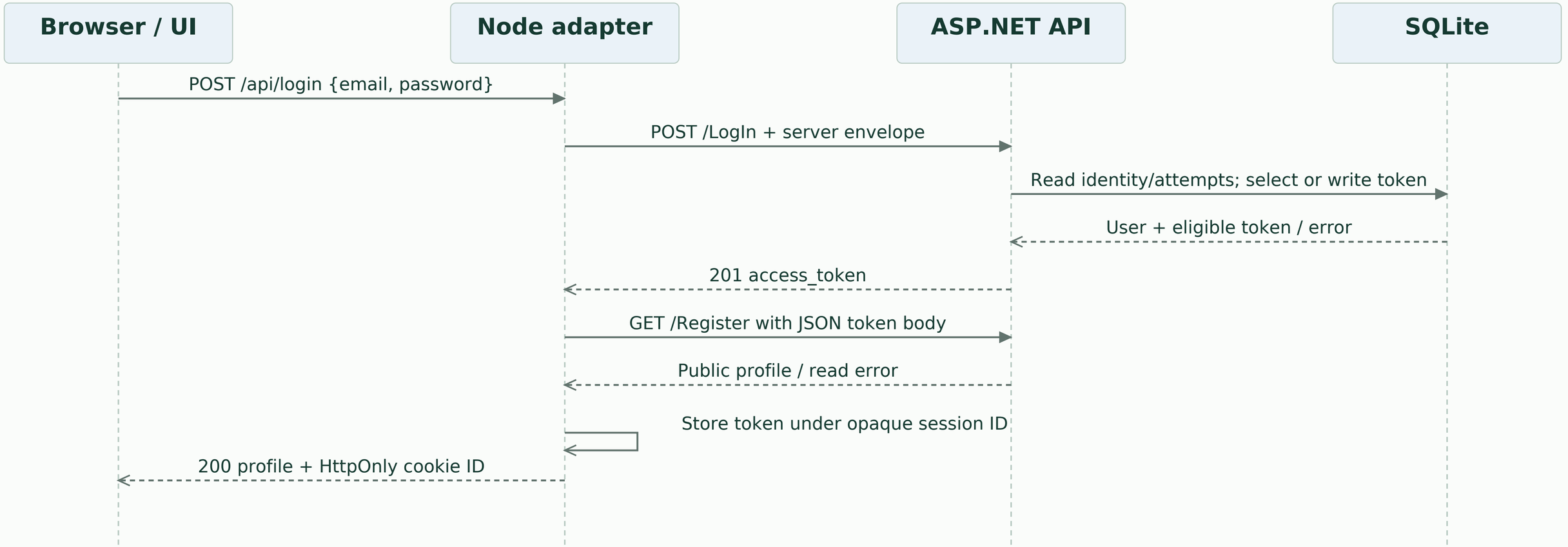
These are actual C# request types, not invented service/entity classes



The inherited access_token is the acting user token; admin_token is a separate service forwarding token.

UML sequence: login establishes a website session

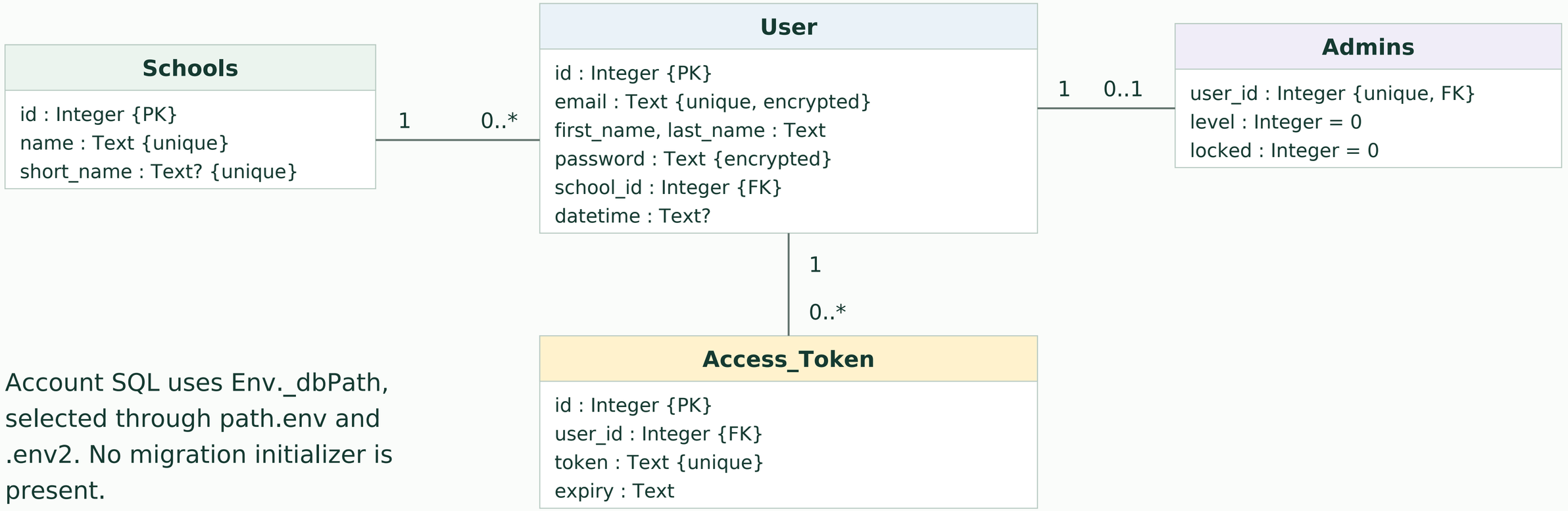
Prepared forwarding identity is assumed; its renewal rules are covered in the appendix



A backend login response alone is insufficient; profile projection succeeds before Node creates the browser session.

Database core: one school, many users, optional admin membership

UML-style relational model of four declared tables; these are not ORM entity classes

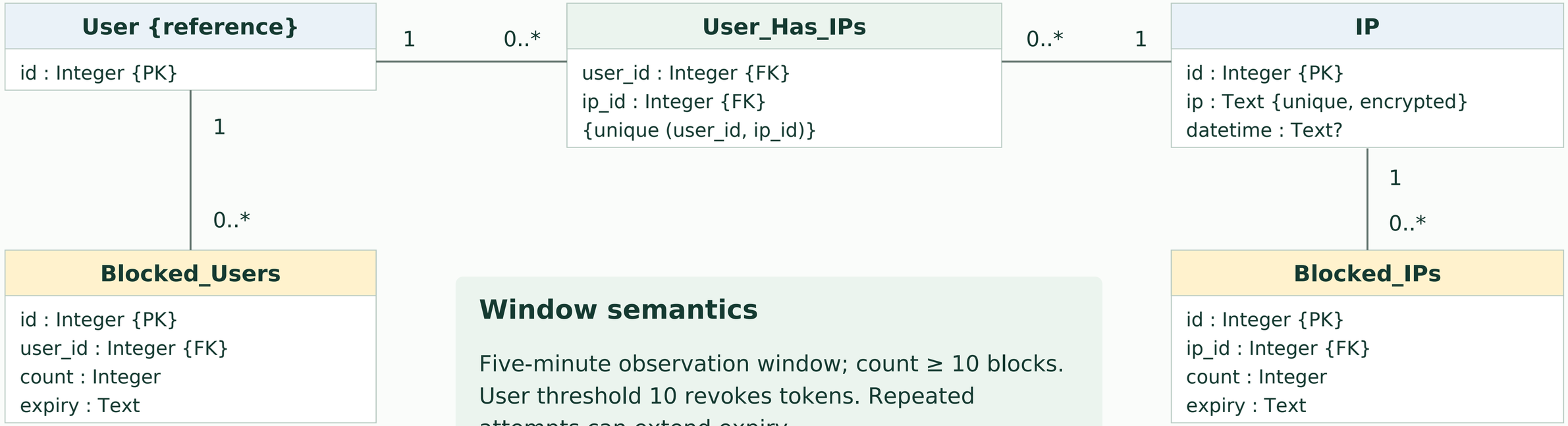


Account SQL uses Env._dbPath, selected through path.env and .env2. No migration initializer is present.

Declared constraints explain intended relationships; actual foreign-key enforcement still needs a runtime check.

Security data: attempts are observations, not always blocks

Four more declared tables track IP identity, user/IP association and attempt windows



Window semantics

Five-minute observation window; count ≥ 10 blocks. User threshold 10 revokes tokens. Repeated attempts can extend expiry.

Token records, attempt windows and process-local lock metadata have different purposes and retention rules.

Listing integration: two contracts currently stop short of a vertical slice

The visible catalog and the direct listing API are separate implementations

Website contract today

Listing[] constant: id, title, category, price, condition, school, cover, label. App holds message/offer strings. No listing fetch route.

gap

API contract today

GET/POST/DELETE /api/Listing. SQL expects title, description, price, category, user_id, datetime. UserId comes from the request body.

Missing connection	Concrete next implementation
Schema + storage	Versioned Listings DDL; align with the account database path.
Identity + authorization	Derive owner from authenticated context; enforce delete ownership.
Adapter + UI	Agree a DTO, map fields and replace sample data with real reads/writes.

Website/data/listings.ts, App.tsx, server/api.mjs; Backend/Listing.cs; DB_Schema.txt

Three identities and two expiry clocks

Browser session, visitor token and service identity serve different purposes

Data item	Held / used where	Meaning
se_session	Browser cookie ↔ Node Map	Random opaque ID; fixed 12-hour local session.
access_token	Node memory → backend JSON	Visitor or human-admin identity; backend 21/3-day lifetime.
admin_token	Node service cache → backend envelope	Login-issued service-admin proof for IP forwarding.
to_user + sudo_id	Backend selected-user context	Target selector plus preserved original caller provenance.

Restart / scaling	Node restart loses local sessions; independent replicas need shared session state.

Service privilege transports identity; it does not turn an ordinary visitor into a human administrator.

api.mjs sessions/forwarding client; AppAuthJson.ValidateApp/ValidateToken; Env.cs

Technology decisions: constraints, benefits and trade-offs

Protocol compatibility, client state and single-instance storage shape the design

Choice	Why it fits this implementation	Trade-off / consequence
React 19 + TypeScript + Vite	Interactive hooks/forms; typed client contracts; fast production bundle.	Local state is not persistence; accessibility can regress with JSX changes.
Node.js 24 adapter	Supports upstream GET bodies; retains credentials; one browser origin.	Extra network hop; in-memory sessions; service identity lifecycle.
C# / ASP.NET Core · .NET 9	Typed controllers and existing account/admin validation paths.	Body-credential contract and static-helper coupling need deliberate handling.
SQLite + parameterized SQL	Portable relational prototype with account/school/security relationships.	Versioned schema, one coherent path and verified FK/concurrency behavior.
Render + Azure configuration	Static UI/API adapter separate from backend deployment.	Configuration/health alone does not prove cross-service readiness.

The Node adapter is the most consequential compatibility decision; local state and SQLite also impose explicit operating limits.

Deployed: Azure backend + Render website

Team-confirmed hosting · https://studentexchange.onrender.com · workflow acceptance remains separate

Deployed Render website

https://studentexchange.onrender.com. Node hosts dist/ + /api. Manifest sets one instance and /healthz; hosting confirmed by the team.

Deployed Azure backend

Team confirms the ASP.NET backend is deployed on Azure. Source includes account/admin controllers, configured SQLite and daily cleanup.

Next acceptance evidence

Current service-admin login; schema/path correctness; explicit FK policy; browser + adapter + API smoke test.

Release acceptance: verify one disposable account workflow and one authorized listing workflow with cleanup, then confirm restart behavior and no credential exposure.

/healthz reports process/configuration presence; it does not prove service login, database readiness or marketplace integration.

render.yaml; server/index.mjs, app.mjs; Env.cs; Website/docs/live-validation.md

Verification: current results and historical live evidence

Today’s check uses the pinned application source, local HTTP fixtures and Chromium

116
passed

1
failed

1
live test skipped

2
builds passed

Evidence level	What it establishes	Boundary of this evidence
Current fixtures	Adapter wire contracts, permissions, UI behavior.	Real Azure/Render configuration or production data.
Current local browser	Preview filtering/message/offer; 390 px home layout.	Seller delivery, persistence or full mobile acceptance.
Historical live report	Oct 8/9 account workflows under their then-current contract.	Live success of the Oct 10 service-token migration.

Remaining verification work is specific: repair the failing assertion/markup contract, then test the current deployed identity and listing path.

evidence/test-summary.json, browser-checks.json, build.log, dotnet-build.log; live-validation.md

Major implementation risks: next integration priorities

Severity and proposed accountable roles are review recommendations, not completed fixes

Priority / issue	Concrete action and acceptance	Proposed owner
P0 · listing ownership	Authenticate create/delete; derive owner server-side; negative ownership tests.	Tristan + Andrew
P0 · listing data path/schema	Add a migration; select one intended database; verify relations and constraints.	Tristan + Andrew
P0 · password storage	Replace reversible passwords with salted password hashing; define account transition.	Andrew + Tristan
P1 · live identity contract	Verify configured service user, admin membership and forwarding expiry behavior.	Andrew + Tristan
P1 · operating limits	Verify FK settings; document restart/session loss and single-instance scope.	Andrew

Prioritize coherent persistence, authenticated ownership and current deployed workflow acceptance.

Listing.cs; DB_Schema.txt; DeterministicEncryption.cs; Sql.cs; api.mjs; team role report

Two concrete quality issues found during this review

Fresh checks reveal actionable defects without rewriting the application during presentation work

Accessible catalog name

Observed: 1 App test fails because “Sample listings” is no longer an accessible list name. Proposed fix: restore the intended label and retain a behavior-focused test. Owner proposal: Andrew + frontend team.

Offer amount validity

Observed: min=0.01 with step=1 rejects $20.00; $20.01 succeeds. Proposed fix: step=0.01, positive-value checks and a browser-level amount regression. Owner proposal: Robert Richardson + Andrew.

No app source was changed for this review. The reported build, suite and browser outcomes describe the current implementation.

Use the failing accessibility expectation and whole-dollar offer case as focused regression acceptance tests.

src/App.test.tsx:20; App.tsx listing-grid and offer input; evidence files

Individual contributions since the proposal

Original roles and completed work, confirmed by the team

Team member / original role	Personally completed (team-reported)	Current status
Tristan Roa · Lead / Backend	UML diagrams; backend commenting; Listings endpoint.	Completed artifacts reported.
Robert (Andrew) Quaife · Backend	Remaining backend; website except messaging UI.	Website responsibility expanded beyond original role.
Alex Pyankov · Materials / PowerPoint	Previous PowerPoint presentation.	Prior presentation deliverable reported.
Robert Richardson · Frontend	Messaging UI.	Local UI implemented; server delivery not complete.
Thomas Phommarath · Frontend	Mobile web development underway.	Mobile web work ongoing.

The contribution record is explicit; repository commit authorship alone is not used to assign individual credit.

Current work and proposed next ownership

Thomas’s mobile web work is ongoing; proposed next assignments are labeled for team agreement

Member	Confirmed current work	Proposed next deliverable
Tristan	Not separately specified.	Lead listing schema/auth contract and integration acceptance.
Andrew	Not separately specified.	Connect adapter/UI to authenticated listing storage; hash/contract work.
Alex	Not separately specified.	Finalize review narrative, evidence and milestone contribution records.
Robert Richardson	Not separately specified.	Fix offer amount UX; define per-listing conversation/persistence contract.
Thomas	Mobile web app work ongoing.	Complete account/admin/detail/mobile checks and record results.

Confirm ownership and dates at the next team check-in; do not infer work-in-progress from a job title.

Next steps: three gates before the final presentation

Dependency order is proposed; calendar dates require the final-presentation deadline

Gate 1 · Persisted marketplace

Owner proposal: Tristan + Andrew. Authenticated create/read/delete; one schema/path; website adapter/DTO; owner-derived identity; unauthorized deletion rejected.

Gate 2 · Acceptance evidence

Owner proposal: Andrew + Robert + Thomas. Green regression suite; whole-dollar offer fix; mobile matrix; service-token/deployed smoke test with cleanup.

Gate 3 · Final readiness

Owner proposal: Tristan + Alex + all. Freeze verified build; rehearse account + listing flow; update UML/ER and per-person contribution delta; keep recovery data.

If durable messaging cannot meet the deadline, agree and disclose a preview-only boundary; do not imply delivery that the system cannot perform.

Finish a coherent, tested slice first; add breadth only after its acceptance conditions pass.

Design consistency: one capability across every layer

Requirements, models, routes, storage and evidence must tell the same story

Capability	Model + request path	Data + evidence
Profile edit	Use case → DTO inheritance → PATCH /api/profile → PATCH/GET /Register.	User + Schools; saved-write/readback alternatives tested.
Target administration	Admin actor → fresh permissions → caller token + exact to_user.	Admins.level/locked + selected User; wire/UI fixtures pass.
Marketplace discovery	Browse use case → App state → imported Listing[] filter.	Six bundled samples; local Chromium 6 → 3 textbooks.
Live listing CRUD	Separate ListingController API exists; website mapping missing.	Expected Listings table absent from DDL; final integration gate.

A model is accurate when it explains the current boundary and exposes gaps, rather than concealing them.

Progress review walkthrough and final-demo acceptance

Use a short, truthful walkthrough tied to implementation evidence

Progress review · 2–3 minute walkthrough

1. Show the actual built page and sample filter. 2. Show local messaging/offer preview. 3. Explain the account request/identity path. 4. Show build/test results and the known failures.

Final · acceptance demonstration

Disposable account + persistent listing; correct owner identity; second-browser read; unauthorized delete rejected; authorized cleanup; verified deployed token flow.

Decision to make now: confirm deadline, current task ownership and a realistic messaging persistence boundary.

Final readiness means a repeatable cross-layer workflow with correct persistence and permissions, supported by evidence.

Appendix | route and permission reference

Browser routes translate into current backend body contracts

Browser contract	Backend operation	Permission / detail
POST /api/register; /api/login	POST /Register; POST /LogIn + GET /Register	Register does not log in; Node creates cookie only after profile.
GET/PATCH /api/profile	GET/PATCH /Register	Valid session/user token; PATCH has separate readback.
POST /api/logout	DELETE /LogIn	Idempotent/qualified confirmation; local cleanup.
GET/DELETE /api/admin/users	GET /Users; DELETE /Register	Admin ≥ 10; deletion requires to_user.
GET/PATCH /api/admin/profile	GET/PATCH /Register with to_user	Admin ≥ 10 + lower-rank/locked-target guards.
GET/DELETE /api/admin/blocked	GET/DELETE /Blocked	Admin ≥ 20; email XOR ip_addr for clearing.
POST/PATCH/DELETE /api/admin	Same verbs /Admin	Admin > 0; assigned level below caller; locked requires 30.
GET/POST/DELETE /api/admin/schools	Same verbs /Schools	Admin ≥ 10 for website management.

Website/server/api.mjs, admin.mjs; Backend controllers and AppAuthJson

Appendix | three edge cases that reveal design understanding

Protocol adaptation, saved writes and qualified revocation each require distinct handling

Forwarding token expired

A marked backend rejection precedes the operation. Renew the service login once; share concurrent refresh; preserve visitor token and request fields. Do not replay arbitrary errors.

Write saved; readback fails

PATCH already committed. Return a saved notice with profile:null; preserve current UI and allow reload. Do not repeat a password write automatically.

Logout success is qualified

Backend 200 may mask token-validation failure. Local cookie/session cleanup and a target sign-out request are different claims from proven backend revocation.

Different failure semantics must produce different client behavior; “retry everything” is incorrect.

Appendix | milestone contribution record

Current review delta is team-reported; proposal date and earlier per-milestone dates are unconfirmed

Current review contributor	Completed / reported delta	Evidence basis
Tristan Roa	UML; backend comments; Listings endpoint.	Team statement; matching artifacts exist.
Robert (Andrew) Quaife	Other backend + website except messaging UI.	Team statement; implementation exists.
Alex Pyankov	Previous PowerPoint presentation.	Team statement; exact earlier milestone/date unconfirmed.
Robert Richardson	Messaging UI.	Team statement; App message handlers/browser preview.
Thomas Phommarath	Mobile web development remains underway.	Team statement; narrow home-page browser check only.

At every later milestone, record an artifact/test result per person—or explicitly record no new completed artifact.

Team responses in this conversation; CONTRIBUTIONS.csv; repository comparison

Appendix | evidence, limitations and presentation handoff

Evidence sources, verified hosting and remaining planning confirmations

Verified here

Current source; frontend and backend builds; 116/1/1 test result; local UI/filter/preview checks; eight declared tables; selected UML/architecture consistency.

Team supplied

Five names/roles; Tristan’s UML/comments/endpoint; Andrew’s backend/website scope; Alex’s prior deck; Robert’s messaging UI; Thomas’s ongoing mobile web work; Azure/Render deployment and the website URL.

Still to confirm

Original proposal artifact/date and approved changes; final deadline; most current assignments; deployed revision and full workflow acceptance.

Presenter handoff: each member owns their contribution statement; technical claims remain linked to source and check evidence.